Un modelo de datos documental en MongoDB para integrar secuencias, taxonomía y perfiles de k-mers de elementos transponibles a través de reinos

Antecedentes

Los elementos transponibles (*transposable elements*, TEs) son secuencias de ADN capaces de moverse o copiarse dentro de un genoma. Lejos de ser "ADN basura", ocupan una fracción muy grande de muchos genomas eucariotas — más del 80% en algunas plantas — y son un motor conocido de evolución, de regulación génica y de adaptación de las especies. Desde el trabajo seminal de Wicker et al. , los TEs se organizan en una clasificación jerárquica de tres niveles — Clase, Orden y Superfamilia — basada en su mecanismo de transposición y su estructura. Esta clasificación conceptual es independiente de la *taxonomía* de la especie en la que se encontró cada elemento (Reino, Filo, Clase, Orden, Familia, Especie), y ambas jerarquías comparten nombres de rango ("Clase", "Orden") sin estar relacionadas entre sí — una fuente frecuente de confusión al modelar estos datos.

Los grupos de investigación en bioinformática trabajan con *bibliotecas* de TEs: colecciones de miles de secuencias de muchas especies, generadas tanto por herramientas automáticas como RepeatModeler2 como por curación experta en bases de referencia como Dfam . Para analizarlas, entrenar clasificadores o comparar especies, estas secuencias suelen transformarse en *perfiles de k-mers*: el conteo de cada subsecuencia de longitud k presente en una secuencia dada. El reto no es solo computar estos perfiles, sino **almacenarlos de forma que sigan siendo consultables**: para $k = 6$ existen $4^6 = 4096$ k-mers posibles por secuencia, y una base con cientos de miles de secuencias puede generar perfiles para decenas de millones de combinaciones k-mer/secuencia en total.

En este trabajo diseñamos, implementamos, poblamos, consultamos y evaluamos una base de datos documental en MongoDB que integra las secuencias, su clasificación de TE, la taxonomía de su especie de origen y sus perfiles de k-mers de $k = 1$ a $k = 6$, a partir de la base pública PanTEon . El objetivo no es un hallazgo biológico nuevo, sino demostrar y justificar un conjunto de decisiones de diseño de datos — qué se embebe, qué se referencia, cómo se representan dos jerarquías con nombres de campo que se prestan a confusión, y cómo se almacenan perfiles de k-mers dispersos de forma eficiente — y respaldar esas decisiones con medidas propias de rendimiento y almacenamiento, no solo con intuición.

Trabajo relacionado

Las bases de datos de TEs de referencia difieren tanto en qué almacenan como en cómo lo organizan. **Repbase** es una de las colecciones más antiguas de secuencias consenso de elementos repetitivos eucariotas; se distribuye como archivos de secuencias anotadas, agrupados por grupo taxonómico, y su clasificación usa nombres propios que no siempre coinciden con el esquema de Wicker et al. . **Dfam** organiza los TEs por *familias*: cada familia tiene una secuencia consenso y un modelo de perfil (HMM), anotaciones sobre genomas y un vínculo a la taxonomía de las especies donde se encuentra, e incluye tanto familias curadas por expertos como familias generadas automáticamente. En ambas, la unidad central es la familia de TE y no la secuencia individual, y ninguna almacena perfiles de k-mers.

PanTEon , la fuente de este trabajo, reúne bibliotecas de TEs de animales, plantas y hongos —provenientes de Dfam o generadas con RepeatModeler2 y MCHelper— con un objetivo

distinto: servir como conjunto de datos para diseñar y entrenar clasificadores de TEs. Se distribuye como dos archivos planos: un FASTA cuyas secuencias llevan la clasificación Clase/Orden/Superfamilia y la especie en el encabezado, y un CSV con la taxonomía y el origen de cada biblioteca. Esa forma es cómoda para distribuir los datos, pero no permite consultarlos directamente: cruzar una secuencia con la taxonomía de su especie, filtrar por reino o recuperar el perfil de k-mers de un elemento exige leer y procesar los archivos completos cada vez. El presente trabajo parte de esa distribución y la convierte en una base de datos consultable, en la que los perfiles de k-mers —la representación que usan los clasificadores— quedan almacenados junto a cada secuencia.

Métodos

Datos y problemas de calidad encontrados

Se usaron las dos fuentes principales de PanTEon Database v1.6.1 (Zenodo, licencia CC BY 4.0): un archivo FASTA con 239 946 secuencias de elementos transponibles (encabezado `>ID#CLASE/ORDEN/SUPERFAMILIA @Especie`) y un archivo CSV con metadatos taxonómicos de 2790 especies (Kingdom, Phylum, Class, Order, Family, Species, Source, Source 2, Notes). Antes de diseñar el modelo se exploraron ambas fuentes por completo (no por muestreo), lo que permitió documentar varios problemas de calidad reales:

- **Mayúsculas/minúsculas mezcladas entre secuencias.** Cada secuencia individual usa consistentemente mayúscula o minúscula, pero el archivo combina ambos estilos entre registros distintos; se normalizó todo a mayúsculas al leer.
- **Alfabeto de nucleótidos.** Una vez normalizado el caso, el alfabeto observado es estrictamente $\{A, C, G, T, N\}$, sin otros códigos IUPAC de ambigüedad. Aproximadamente 35% de las secuencias (muestra de 20 000) contienen al menos una N, en promedio ~1.1% de su longitud en las que la tienen, con un caso extremo cercano al 50%.
- **Valores faltantes codificados como texto.** La columna `Order` del CSV usa, además de valores vacíos, cadenas de texto como `"No trobat"` (catalán, "no encontrado"), `"No order"`, `"Not order identified <Clase>"` o frases con `"incertae sedis"` para representar lo mismo que un valor nulo. Al normalizar estas cadenas a `null`, los nulos de `Order` pasaron de 29 a 55.
- **Nombres de clado usados como "especie".** 45 filas del CSV (todas con `Source = Dfam`) tienen en la columna `Species` un taxón superior en vez de un binomio — por ejemplo `Primates`, `Rodentia` o `Mammalia` — correspondientes a bibliotecas de consenso ancestral a nivel de clado, no a especies individuales.
- **Cruce FASTA↔CSV.** El FASTA y el CSV contienen exactamente 2790 especies cada uno, pero 4 no cruzaban directamente por un problema de puntuación: el FASTA usa abreviaturas con punto (`subsp.`, `sp.`) y el CSV las mismas sin punto. Normalizando la llave de cruce (quitando puntos, minúsculas, espacios colapsados) las 2790 especies cruzan exactamente, sin huérfanas en ningún lado.

Modelo de datos

El modelo (Fig. 1) usa dos colecciones principales. **`species`** (2790 documentos) almacena la taxonomía completa de cada especie y la procedencia de su biblioteca de TEs (`source`, `source2`, `notes`). **`sequences`** (239 946 documentos) almacena, por cada elemento transponible: su identificador original (`original_id`), su clasificación de TE (`te_class`, `te_order`, `te_superfamily`), una referencia a su especie (`species_id`) más una copia denormalizada de `species_name`, la secuencia de nucleótidos, tres atributos derivados (`length`, `gc_content`, `n_percentage`) y los perfiles de k-mers embebidos en `kmers.k1`…`kmers.k6`.

Modelo de datos. Dos colecciones de producción (`species`, `sequences`) relacionadas por referencia, y una colección auxiliar (`kmer_profiles_alt`) usada solo en el experimento descrito en .

Cuatro decisiones de diseño merecen justificación explícita:

Dos colecciones, no una. Se consideró una única colección de `species` con los TEs embebidos como arreglo dentro de cada especie, lo que evitaría cualquier cruce. Se descartó por el límite de 16 MB por documento de MongoDB: en el experimento, el tamaño lógico medio de una secuencia con sus k-mers fue de unos 38 KB (1128 MB para 30 000 secuencias), de modo que los 1627 TEs de *Malus domestica*, la especie con más elementos, sumarían alrededor de 61 MB, casi cuatro veces el límite. Separar `species` y `sequences` evita ese tope y permite consultar secuencias sin cargar la especie completa.

Dos jerarquías, campos planos con nombres explícitos. La clasificación del TE (Clase/Orden/Superfamilia) y la taxonomía de la especie (Reino → Especie) comparten nombres de rango. Se consideró agruparlas en un subdocumento genérico del tipo `classification: {class, order, …}`, pero se descartó porque podría usarse por error para cualquiera de las dos jerarquías. Se usaron campos planos distintos (`te_class`, `te_order`, `te_superfamily` frente a `taxo_class`, `taxo_order`), lo que además permite indexarlos directamente.

Referencia, no embebido, entre secuencia y especie. Se consideró embeber la taxonomía completa en cada secuencia, pero se descartó: la taxonomía se repetiría en promedio 86 veces por especie (239 946/2790), y una especie con la corrección de un dato taxonómico obligaría a actualizar todas sus secuencias. Se referencia por `species_id`, con `species_name` denormalizado como única excepción — un campo pequeño y estable, necesario para mostrar resultados sin pagar un `$lookup` en cada consulta simple.

K-mers embebidos como diccionario disperso, por conteo. Se almacenan conteos (no frecuencias, que se derivan dividiendo por $longitud - k + 1$) y solo las combinaciones observadas, no un arreglo denso de 4^k posiciones: para $k = 6$ eso implicaría 4096 entradas mayormente en cero por documento. Los k-mers se embeben junto a su secuencia, no en una colección aparte, porque nunca se consultan sin ella ni se comparten entre secuencias (ver para la comparación cuantitativa con la alternativa). Los k-mers que contienen cualquier carácter distinto de A/C/G/T se excluyen de los conteos, no se rellenan ni se tratan como un símbolo adicional del alfabeto.

Se aplicó validación de esquema (`$jsonSchema`) en ambas colecciones — campos obligatorios, tipos, y en `sequences.sequence` un patrón `^[ACGTN]+$` que impide insertar caracteres fuera del alfabeto esperado — verificada insertando deliberadamente un documento inválido. Los índices creados y la consulta que justifica cada uno se listan en la Tabla [tab:indices].

@p3.4cmp2.0cmp6.2cm@ Índice & Colección & Consulta que soporta
`species_name` (único) & species & Unicidad; cruce por nombre
`match_key` & species & Cruce FASTA↔CSV durante la carga
`kingdom` & species & Agrupar/filtrar por reino
`original_id` (único) & sequences & Búsqueda por identificador; unicidad
`species_id` & sequences & `$lookup` con species; conteos por especie
(`te_class`, `te_order`, `te_superfamily`) & sequences & Conteos por clasificación (consulta 1)
(`species_name`, `te_superfamily`) & sequences & Consulta 2 (una especie), consulta 9 y biblioteca de una especie (consulta 10)

Pipeline en Python

El pipeline (Python 3.14, `pymongo`, `pandas`) procesa el FASTA en *streaming* línea por línea (sin cargarlo completo en memoria, dado su tamaño de ~990 MB), valida cada encabezado contra el patrón esperado, limpia el CSV conservando solo las nueve columnas requeridas, normaliza nombres de especie, calcula los perfiles de k-mers de $k = 1$ a 6 y dos atributos adicionales por secuencia (`gc_content`, `n_percentage`), y carga los documentos en MongoDB con `bulk_write` en lotes de 200. La carga completa de las 239 946 secuencias tomó 1778.7 s (~29.6 min) en un equipo de escritorio estándar, sin omitir ninguna

secuencia tras la normalización del cruce FASTA↔CSV. El pipeline es reproducible: cualquier persona puede regenerar la base desde cero ejecutando tres scripts documentados en el repositorio sobre los dos archivos descargados de Zenodo.

Diseño de los experimentos

Para evaluar la decisión de embeber los k-mers se comparó esa estrategia contra una **colección aparte** (metadatos de la secuencia en una colección, k-mers en otra, referenciados por `sequence_id`), sobre una muestra de 30 000 secuencias *ya cargadas* — es decir, con los k-mers ya calculados — para aislar el costo de la estrategia de almacenamiento del costo de calcular los k-mers. La muestra es reproducible: los identificadores se eligen al azar con una semilla fija. Para cada estrategia se midió el tiempo de carga (`bulk_write`), el espacio en disco (`collStats`) y tres consultas representativas: (Q1) el perfil de 6-mers de una secuencia, (Q2) los perfiles de 6-mers de todas las secuencias de una especie y (Q3) los metadatos junto con el perfil completo de una secuencia. Cada consulta se ejecutó con y sin índice; para la versión sin índice, los índices secundarios se ocultaron con `collMod` (`hidden: true`), lo que obliga al planificador a recorrer la colección sin tener que borrar ni reconstruir el índice. Cada consulta se analizó con `explain` en verbosidad `executionStats` (plan elegido, documentos y llaves examinados, tiempo en el servidor) y se midió además su tiempo desde el cliente; se reporta la mediana de tres repeticiones.

Uso de herramientas de inteligencia artificial

Siguiendo la política de BMC sobre modelos de lenguaje, se declara que el equipo usó un asistente de inteligencia artificial (Claude, de Anthropic) como apoyo en una parte pequeña del código y en la creación de las figuras. Todos los resultados reportados provienen de ejecutar ese código sobre la base de datos del proyecto; las cifras de las tablas y las figuras se verificaron contra las salidas de los notebooks del repositorio. Los autores revisaron el contenido y asumen la responsabilidad del trabajo.

Resultados

Composición de la base

Tras la carga completa, `sequences` contiene 239 946 documentos y `species` 2790, sin IDs duplicados ni referencias `species_id` huérfanas (verificado con una agregación `$group`/`$count` y un `$lookup` de control). La Tabla 1 resume la composición por clase y orden de TE (consulta 1). CLASSI (retrotransposones) domina con 194 774 TEs (81.2%) frente a 45 172 (18.8%) de CLASSII (transposones de ADN); LTR es el orden más abundante (119 368), seguido de LINE (68 382).

Cantidad de TEs por clase y orden (consulta 1).

Clase	Orden	Cantidad
CLASSI	LTR	119 368
CLASSI	LINE	68 382
CLASSII	TIR	36 471
CLASSII	HELITRON	5 075
CLASSI	SINE	3 634
CLASSII	MITE	3 456
CLASSI	DIRS	2 933
CLASSI	PLE	344
CLASSI	RETROPOSON	113
CLASSII	CRYPTON	112
CLASSII	MAVERICK	58
Total CLASSI		**194 774**
Total CLASSII		**45 172**

Entre reinos (consulta 3), Animalia está dominado por LINE (44 477 TEs) sobre LTR (25 382), mientras que Plantae y Fungi están dominados por LTR — visible de un vistazo en la Fig. 2, que agrega los 11 órdenes para los 3 reinos a partir de una agregación con `$lookup`. Las 10 especies con más TEs están encabezadas por *Malus domestica* (1627), *Danio rerio* (1352) y *Callithrix jacchus* (1294) (consulta 4); en el otro extremo, varias decenas de especies tienen exactamente 1 TE registrado, consistente con bibliotecas de anotación preliminar más que con genomas realmente pobres en TEs.

Composición de elementos transponibles por orden y reino. Cantidad absoluta de TEs de cada orden en Animalia, Plantae y Fungi, calculada con una agregación `$lookup` + `$group` sobre las 239 946 secuencias.

Longitud y contenido GC

CLASSI es, en promedio, más del doble de largo que CLASSII (4453.1 nt frente a 2137.3 nt; consulta 5), con longitud mínima de 81 nt y máxima de 19 926 nt en toda la base. La Fig. 3 muestra la distribución completa por orden: MAVERICK y LTR tienen la mediana más alta y la mayor dispersión, mientras que SINE y MITE son consistentemente cortos. El contenido GC promedio (consulta 6) es más alto en Fungi (46.2%) y más bajo en Plantae (41.2%) (Fig. 4); entre órdenes, SINE y DIRS tienen el GC% más alto (~53%) y TIR/MAVERICK el más bajo (~40%). El TE más largo de toda la base (19 926 nt, orden LTR/GYPSY) pertenece a *Helianthus annuus*; los más cortos (81–100 nt) aparecen en los tres reinos (consulta 7).

Distribución de la longitud de los TEs por orden. Cajas construidas con cuartiles calculados en MongoDB (`$percentile`, P5–P95), sin necesidad de extraer los datos crudos.

Contenido GC por reino (P5–P95).

Origen de los datos y control de calidad

Agrupando por la columna `Source` del CSV (consulta 13), las bibliotecas generadas automáticamente con RepeatModeler2 + MCHelper (`RM2+MCH`) cubren 2404 especies con 208 354 secuencias (86.7 secuencias/especie en promedio, longitud media 4375.5 nt), mientras que las bibliotecas de referencia Dfam cubren 386 especies con 31 592 secuencias (81.8 secuencias/especie, longitud media 1653.8 nt) — una diferencia de longitud notable, consistente con que Dfam incluye más fragmentos de consenso cortos y las 45 bibliotecas

de clado ancestral descritas en Métodos. Como consulta de control de calidad propia (consulta 14), se identificaron 4374 secuencias (1.82% del total) con más de 5% de contenido N, posible indicio de regiones de ensamblaje genómico incompleto; estas secuencias se conservan en la base (no se descartan), y el pipeline simplemente excluye de los conteos cualquier k-mer que contenga una N.

Patrones de k-mers

La Fig. 5 muestra la frecuencia relativa promedio de los 16 2-mers para cada orden de TE (consulta propia, visualización 4). Casi todos los órdenes comparten el mismo patrón dominante — AA/TT/AT altos, CG consistentemente el más bajo, el conocido déficit de dinucleótidos CpG de los genomas eucariotas — mientras que RETROPOSON y SINE se distinguen por tener CC y GG relativamente más altos que el resto. Recuperar el perfil completo de k-mers de una secuencia dada para un *k* dado (consulta 15) es una búsqueda puntual por `original_id` seguida de una proyección de `kmers.k{k}`; para `PDB00000001` (HELITRON de *Halicephalobus mephisto*, el ejemplo del enunciado del taller, recuperado en la consulta 8 junto con su taxonomía completa vía `$lookup`) el perfil de 2-mers tiene presentes las 16 combinaciones posibles, con AA (306) y TT (234) como las más frecuentes.

Frecuencia relativa promedio de 2-mers por orden de TE. Cada celda es el promedio, sobre todas las secuencias de ese orden, del conteo de un dinucleótido dividido entre *longitud* − 1 (en las secuencias con N, los 2-mers que contienen N no se cuentan).

Evaluación experimental: embebido frente a colección aparte

Sobre la muestra de 30 000 secuencias (Tabla 2), la carga embebida tomó 9.7 s frente a 13.5 s de la colección aparte, que hace el doble de escrituras y mantiene un índice adicional. El tamaño lógico de los datos fue prácticamente el mismo en ambas estrategias (1128 MB frente a 1129 MB sin comprimir), pero el espacio real en disco fue menor en la colección aparte (372 MB frente a 507 MB): al separar los k-mers, cada colección guarda documentos más homogéneos y la compresión de WiredTiger es más efectiva.

El índice resultó mucho más determinante que la estrategia de almacenamiento. **Con índice**, las tres consultas usaron un plan `IXSCAN` y examinaron solo los documentos que devuelven; las consultas puntuales Q1 y Q3 tardaron menos de un milisegundo en ambas estrategias. **Sin índice**, todas cayeron en un `COLLSCAN` que examinó la colección completa (30 000 documentos en el modelo embebido y 60 000 en la colección aparte, que recorre dos colecciones), y Q1 pasó de 0.33 ms a 19.2 ms en el modelo embebido —unas 58 veces más lenta— y de 0.64 ms a 29.2 ms en la colección aparte. En Q2, el tiempo en el servidor pasó de 1 ms a 24–37 ms al quitar el índice; en el cliente la diferencia es menor (41 ms frente a 67 ms en el modelo embebido) porque la mayor parte del costo está en transferir y decodificar los 179 perfiles de 6-mers devueltos, no en encontrarlos.

Comparación embebido vs. colección aparte (muestra de 30 000 secuencias). Tiempos medidos desde el cliente (mediana de 3 repeticiones); entre paréntesis, documentos examinados según `explain("executionStats")`.

Métrica	Embebido	Colección aparte
Tiempo de carga (s)	9.67	13.50
Espacio en disco (MB)	506.59	372.26 (43.90 + 328.37)
Tamaño lógico sin comprimir (MB)	1128.02	1129.43
Índices (MB)	1.00	2.00
Q1 perfil de una secuencia, con índice (ms)	0.33 (1)	0.64 (2)
Q1 perfil de una secuencia, sin índice (ms)	19.18 (30 000)	29.23 (60 000)
Q2 perfiles de una especie, con índice (ms)	41.38 (179)	48.14 (358)
Q2 perfiles de una especie, sin índice (ms)	66.58 (30 000)	81.46 (60 000)
Q3 metadatos + perfil, con índice (ms)	0.42 (1)	0.74 (2)
Q3 metadatos + perfil, sin índice (ms)	18.97 (30 000)	29.62 (60 000)
Suma global de un 6-mer, agregación (ms)	337.31 (30 000)	327.02 (30 000)

En las tres consultas, la colección aparte necesitó dos consultas secuenciales donde el modelo embebido usó una, y fue algo más lenta tanto con índice como sin él. La única operación en la que la colección aparte fue ligeramente más rápida es la agregación global sobre un k-mer (327 ms frente a 337 ms), que lee todos los documentos y no se beneficia de ningún índice.

Discusión

Los resultados respaldan la decisión de diseño de embeber los k-mers junto a su secuencia: dado que nunca se consultan de forma aislada ni se comparten entre secuencias, una colección aparte solo añade una referencia y un segundo *round-trip* sin beneficio claro para este patrón de acceso, y con el índice correcto el modelo embebido fue el más rápido en las tres consultas medidas. A cambio, ocupa más espacio en disco: la colección aparte se comprimió mejor y ahorró cerca de una cuarta parte del almacenamiento. Esa ventaja, junto con su leve ventaja en la agregación global, solo pesaría en un escenario distinto al de este proyecto: un servicio que agrega k-mers con mucha frecuencia *sin* necesitar nunca la secuencia ni sus metadatos. El experimento también muestra que la decisión más importante no es dónde guardar los k-mers sino qué índices crear: sin ellos, cualquiera de las dos estrategias recorre la colección completa y las consultas puntuales se vuelven entre 40 y 60 veces más lentas.

La exploración de calidad de datos fue, en sí misma, una parte sustancial del trabajo de diseño. Los valores faltantes codificados como texto libre en distintos idiomas (catalán e inglés mezclados en la misma columna) y el uso de nombres de clado como si fueran nombres de especie son el tipo de inconsistencia que un modelo relacional con tipos estrictos habría forzado a resolver antes de poder cargar los datos; en un modelo documental sin `$jsonSchema` estricto en todos los campos, estos problemas pueden pasar desapercibidos silenciosamente. Aplicar validación de esquema solo en los campos verdaderamente estructurales (identificadores, secuencia, tipos numéricos) — y dejar los campos taxonómicos más flexibles (`string` o `null`) — fue un compromiso deliberado entre seguridad de datos y la naturaleza real, incompleta, de los metadatos taxonómicos disponibles.

Limitaciones. El experimento de almacenamiento se corrió sobre una muestra de 30 000 secuencias (12.5% de la base), no sobre las 239 946 completas, por restricciones de tiempo. El umbral de 5% de contenido N usado en el control de calidad es una elección heurística propia, no un estándar de la literatura. Finalmente, el cruce de especies se resolvió por normalización de texto (minúsculas, sin puntos); una especie con un error ortográfico genuino en una sola fuente no sería detectada por este método y quedaría, silenciosamente, sin cruzar.

Amenazas a la validez. *Validez interna:* todas las mediciones se hicieron en un solo

equipo de escritorio, con MongoDB y el cliente en la misma máquina y otros procesos activos; los tiempos de milisegundos son sensibles a ese ruido, por lo que se reporta la mediana de tres repeticiones y las conclusiones se apoyan sobre todo en métricas que no dependen de la máquina, como los documentos examinados y el plan elegido. Las consultas se midieron con la caché de WiredTiger ya caliente (la muestra de ~1.1 GB cabe en memoria), así que los tiempos sin índice son los de un recorrido en memoria; con la caché fría, leyendo desde disco, serían mayores y la diferencia con la versión indexada crecería. Además, el espacio en disco medido con `storageSize` varió entre ejecuciones (la colección aparte ocupó entre 294 y 372 MB en distintas corridas) porque incluye espacio reservado por el motor; el tamaño lógico, en cambio, fue estable. *Validez externa:* la muestra de 30 000 secuencias puede no reflejar el comportamiento sobre la base completa, en especial cuando los datos dejen de caber en memoria, y los resultados corresponden a una versión concreta de MongoDB (9.0) y a su configuración por defecto. *Validez de constructo:* las tres consultas elegidas representan los patrones de acceso de este proyecto —búsqueda puntual, búsqueda por especie y lectura combinada de metadatos y k-mers—; para otras cargas de trabajo, como agregaciones masivas sobre k-mers, la conclusión podría ser distinta, tal como sugiere la consulta de referencia sobre la suma global.

Si los datos crecieran 10 o 100 veces, el diseño actual seguiría siendo válido en su estructura, pero dos decisiones deberían revisitarse: (1) el cálculo de k-mers en Python puro (sin paralelismo) tardó cerca de 30 minutos para 240 mil secuencias; a 24 millones de secuencias ese tiempo sería inviable sin paralelizar el pipeline o delegar el cómputo a una extensión nativa; y (2) aunque en el experimento de la sección de Resultados la colección aparte solo mostró ventaja en espacio en disco y no en tiempo de consulta, a una escala donde la colección `sequences` no quepa cómodamente en el conjunto de trabajo en memoria de MongoDB, separar el campo más pesado (la secuencia) de los campos consultados con más frecuencia podría volver a ser ventajoso — una pregunta que este experimento, a la escala actual, no puede responder por sí solo.

Conclusiones

Se diseñó, implementó y evaluó un modelo de datos documental en MongoDB que integra 239 946 secuencias de elementos transponibles, su taxonomía y sus perfiles de k-mers de $k = 1$ a 6, resolviendo las quince consultas obligatorias y dos preguntas de investigación propias mediante agregaciones. La decisión de embeber los k-mers junto a su secuencia, en vez de separarlos en una colección aparte, se justificó no solo conceptualmente sino con una medida experimental propia: con el índice adecuado, el modelo embebido resolvió las consultas puntuales en menos de un milisegundo con una sola consulta, mientras que la colección aparte necesitó dos y solo ofreció a cambio un menor espacio en disco. Sin índice, ambas estrategias fueron entre 40 y 60 veces más lentas en búsquedas puntuales — evidencia directa de que los índices sobre `original_id` y `species_id`, y no la separación en colecciones, son lo que realmente sostiene el rendimiento de este modelo. La exploración sistemática de ambas fuentes de datos, antes de fijar el esquema, permitió documentar y resolver problemas de calidad reales (nombres inconsistentes, valores faltantes disfrazados de texto, nombres de clado usados como especie) que de otro modo habrían quedado embebidos silenciosamente en el modelo final.

Lista de abreviaturas

BSON	*Binary JSON*, formato de documentos de MongoDB
CSV	Valores separados por comas (*comma-separated values*)
DIRS	Retrotransposones de tipo *Dictyostelium intermediate repeat sequence*
GC	Contenido de guanina y citosina
HMM	Modelo oculto de Markov (*hidden Markov model*)
LINE	Elemento nuclear largo intercalado (*long interspersed nuclear element*)
LTR	Repetición terminal larga (*long terminal repeat*)
MITE	Elemento transponible pequeño con repeticiones invertidas
nt	Nucleótidos
RM2+MCH	Bibliotecas generadas con RepeatModeler2 y MCHelper
SINE	Elemento nuclear corto intercalado (*short interspersed nuclear element*)
TE	Elemento transponible (*transposable element*)
TIR	Repeticiones terminales invertidas (*terminal inverted repeats*)

Declaraciones

No aplica.

No aplica.

Los datos analizados en este estudio están disponibles en el repositorio Zenodo de PanTEon Database v1.6.1 , bajo licencia CC BY 4.0. El código del pipeline, las consultas, los experimentos y las figuras está disponible en el repositorio del proyecto , que documenta cómo descargar los datos y reconstruir la base desde cero; el repositorio no redistribuye los archivos de datos originales.

- Nombre del proyecto: PanTEon-Mongo
- Página del proyecto: https://github.com/Teoqm/panteon-bd2-elementos-transponibles
- Sistema operativo: independiente de la plataforma (probado en Windows 11)
- Lenguaje de programación: Python
- Otros requisitos: Python 3.11 o superior, MongoDB Community Server, pymongo, pandas, numpy, matplotlib, seaborn, jupyter
- Licencia: no especificada
- Restricciones de uso para no académicos: ninguna

Los autores declaran que no tienen intereses en conflicto.

No aplica.

TRC analizó las fuentes FASTA y CSV, realizó la limpieza de los datos y el informe de calidad, y diseñó el modelo documental de MongoDB, sus índices y validaciones. JDGT implementó el pipeline en Python, el cálculo de los perfiles de k-mers y la carga en MongoDB, desarrolló las consultas y generó las bibliotecas FASTA. MQM diseñó y ejecutó la evaluación experimental de las estrategias de almacenamiento, construyó las visualizaciones y coordinó la redacción del artículo. Todos los autores leyeron y aprobaron el manuscrito final.

Este trabajo se realizó como proyecto del curso Bases de Datos II de la Universidad Autónoma de Manizales, bajo la orientación del docente Simón Gaviria Orrego.

Agradecemos a los autores de PanTEon Database por hacer públicos los datos que hicieron posible este trabajo.